WEX 428 - NLP / Sentiment Analysis Project Report

Fake News Explainability Platform

A web application for classifying Fake vs. Real news articles and visualizing model decisions using LIME and SHAP

Student	ABDIRIZAK HASSAN
Student ID	220201993
University	Ostim Technical University
Date	20 October 2026
Source code	https://github.com/ABDIRIZAK23HASSAN/fake-news-explainability-platform-220201993

Note on reproducibility: All charts, tables, and screenshots in this report come directly from the project repo. The standalone experiment numbers were generated by running scripts/run_experiments.py, the platform model scores came out of the admin training pipeline, and the UI screenshots were captured during an automated Chromium test run.

1. Introduction

If you look at most fake-news classifiers, they just spit out a 'Fake' or 'Real' label and expect you to take their word for it. For this project, I wanted to build a web platform that actually shows what is going on under the hood. When a user pastes an article or uploads a file, the app highlights the exact words that pushed the model toward its verdict, alongside a confidence percentage, the top extracted keywords, and the five closest articles from the reference dataset. I designed it to be a decision-support tool rather than an automatic truth detector. Also, in line with the WEX 428 course rules, no external LLM APIs are called anywhere in the project—every single model is trained locally from scratch.

The main goals I set out to cover were: (1) putting together a clean NLP preprocessing pipeline backed by controlled experiments; (2) training and comparing five different classical ML models; (3) wiring up LIME and SHAP so users can see word-level feature weights; (4) building a full-stack web app with separate user and admin views, JWT authentication, a PostgreSQL database, and a REST API; and (5) packaging everything with Docker Compose so it is easy to spin up and test.

2. Dataset

We worked with the standard two-file dataset (Fake.csv and True.csv). Each CSV has four columns: title, text, subject, and date. Since the class label is just implied by which file a row comes from, I added a binary label column (0 for Fake, 1 for Real) when merging them. Out of the box, the two files add up to 44,898 rows, though a lot of those turned out to be duplicates or blank entries once I started digging into the data.

Stage	Articles	Fake	Real
Raw CSV files	44,898	23,481	21,417
After stripping empty rows and duplicate texts (experiment script)	38,653	17,459	21,194
Imported into the web platform (also drops empty titles/subjects and texts < 5 words)	38,559	17,365	21,194

While cleaning the data, a few quality issues stood out right away. Fake.csv had 6,026 duplicate article bodies and 630 completely blank texts, whereas True.csv only had 225 duplicates. When I ran the files through the platform's strict importer—which also rejects missing titles, missing subjects, or super short texts under 5 words—Fake.csv came down to 17,365 valid rows (773 rejected, 5,343 duplicates skipped) and True.csv landed at 21,194 valid rows (1 rejected, 222 duplicates skipped). I also noticed that the pre-split train.csv and test.csv files given for the course actually shared 82 identical articles between train and test. To prevent that overlap from inflating our test scores, I deduplicated the combined pool first and generated a fresh stratified 80/20 split using seed 42.

Another huge issue in this dataset is label leakage. Almost every single article in True.csv comes from Reuters, meaning the token 'reuter' shows up in 99.8% of Real articles compared to just 1.2% of Fake ones. On top of that, the subject column splits the two classes with zero overlap—Real articles use tags like 'politicsNews' and

'worldnews', while Fake ones use 'News', 'politics', or 'left-news'. If you feed subject or the Reuters dateline into a model, it gets nearly 100% accuracy just by cheating on metadata. Because of that, I completely dropped the subject column from the feature set and wrote a regex filter to strip out source datelines and boilerplate before tokenizing (I documented the full breakdown in docs/leakage_analysis.md).

3. NLP pipeline

To make sure the training pipeline and live user predictions never get out of sync, both import the exact same preprocessing module. When an article comes in, it goes through six steps:

- **Basic text cleaning —** strips out HTML tags and entities, URLs, email addresses, numbers, apostrophes, and punctuation, then lowercases everything and collapses extra whitespace.
- **Removing publisher artifacts —** cuts out the 'WASHINGTON (Reuters) -' style datelines, standalone mentions of 'Reuters', and common clickbait footer/header tags like 'Featured image via', 'Getty Images', and '(VIDEO)'.
- **Tokenization and stop-words —** splits on whitespace, drops any leftover fragments shorter than 2 letters, and filters against a 318-word stop-word list that admins can edit directly from the dashboard.
- **Stemming —** runs words through the English Snowball stemmer, wrapped in a dictionary cache so repeated words do not slow down large batch imports.
- **TF-IDF vectorization —** extracts unigrams and bigrams with sub-linear TF scaling, min_df=3, max_df=0.9, and a cap of 50,000 features for linear models (or 10,000 for the tree-based models so memory usage stays reasonable).
- **Train/test split —** uses a stratified 80% train / 20% test split (seed 42), keeping the 20% test partition strictly locked away until final evaluation.

On top of those features, I trained five classifiers: Logistic Regression (C = 4), Linear SVM (C = 0.5, wrapped in Platt sigmoid calibration so we can get probability estimates out of it), Multinomial Naive Bayes (alpha = 0.1), Random Forest (100 trees), and XGBoost (200 trees, max depth 6). Whatever probability the active model assigns to the winning class is what gets displayed in the UI as the confidence score.

4. Experiments

For the offline experiments, the deduplicated dataset was split into 30,922 training articles and 7,731 test articles. One statistical note worth keeping in mind: at around 98.6% accuracy on a test set of 7,731 items, the standard error is roughly 0.13%. That means any accuracy gap smaller than ~0.3% between two runs could easily just be random split noise rather than a real improvement.

4.1 Preprocessing strategies and label leakage

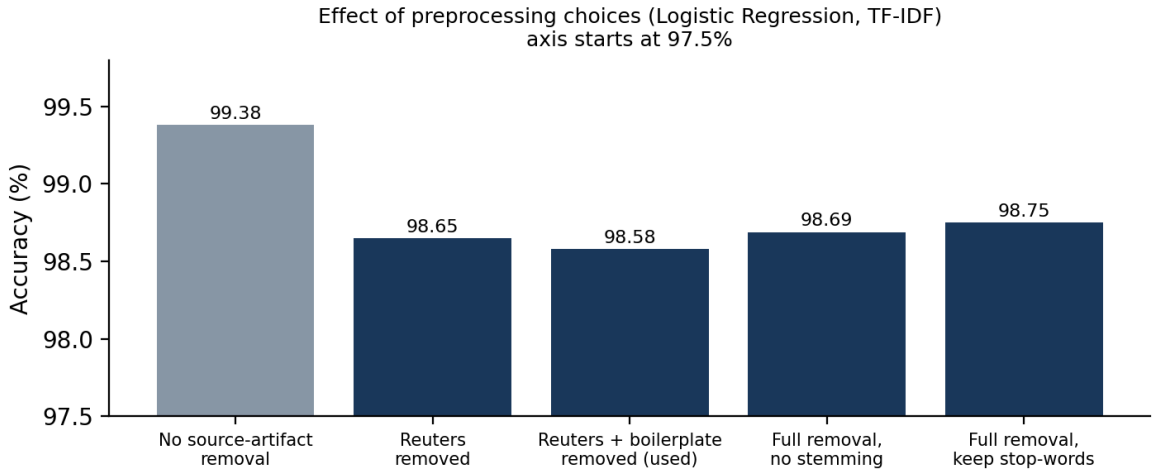


Figure 1. Stripping the Reuters dateline drops accuracy by about 0.7%; toggling stemming or stop-words shifts it by less than 0.2% (well within noise).

Variant	Accuracy	F1
L0_no_leakage_removal (raw cleaned text)	99.38%	99.44%
L1_reuters_removed (datelines + 'reuters' stripped)	98.65%	98.78%
L2_full_removal (Reuters + boilerplate stripped — used in app)	98.58%	98.71%
L2_no_stemming (full removal, stemmer disabled)	98.69%	98.81%
L2_keep_stopwords (full removal, stop-words kept)	98.75%	98.86%

As expected, leaving the publisher artifacts in (L0) gives an artificially high 99.38% accuracy. Once we strip out the Reuters mentions and boilerplate (L2), accuracy settles down to 98.58%. Just to compare, I also tested the instructor's original train/test split: Logistic Regression hit 99.12% out of the box, and simply deleting the single word 'reuter' brought it down to 98.60%. The reason accuracy stays above 98% even without the publisher name is that the two sources have very different writing styles—formal wire-service reporting versus opinionated blog posts—which we can see clearly when we look at the LIME and SHAP weights in Section 6.

4.2 Feature extraction

Feature extraction	Accuracy	F1	Training time
Bag of Words (unigrams)	97.96%	98.15%	8.83 s

TF-IDF (unigrams)	98.58%	98.71%	5.44 s
TF-IDF (unigrams + bigrams)	98.72%	98.84%	28.09 s

Plain Bag of Words lagged behind both TF-IDF setups by a noticeable margin (97.96% vs. 98.58%). Adding bigrams to TF-IDF nudged accuracy up by another 0.14 points to 98.72%, though it took about five times longer to fit (28 seconds instead of 5.4 seconds). Technically, a 0.14% gain is inside the noise margin, but 28 seconds is still plenty fast for a background training job, and having two-word phrases like 'read more' or 'white house' makes the explanations more informative. We went with TF-IDF unigrams + bigrams as the default. We did not test Word2Vec, FastText, or trigrams in this study.

4.3 Model comparison (experiment split)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Train time
Logistic Regression	98.72%	98.25%	99.43%	98.84%	0.9991	28.61 s
Naive Bayes	95.36%	95.86%	95.66%	95.76%	0.9879	26.38 s
Linear SVM (calibrated)	98.97%	98.87%	99.25%	99.06%	0.9993	28.26 s
Random Forest	97.43%	96.98%	98.37%	97.67%	0.9974	86.81 s
XGBoost	98.58%	98.34%	99.08%	98.71%	0.9990	239.21 s

5. Final models trained inside the platform

After building the web app, I used the admin interface to import the CSVs into PostgreSQL and ran the built-in model comparison job across all 38,559 stored articles (30,847 train / 7,712 test). The numbers in the table below vary by a few hundredths of a percent from Section 4.3 simply because the database importer also filters out articles with fewer than 5 words.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Train time
Logistic Regression (ACTIVE)	98.76%	98.32%	99.43%	98.87%	0.9993	46 s
Linear SVM (calibrated)	98.95%	98.89%	99.20%	99.05%	0.9994	32 s
Naive Bayes	95.44%	96.13%	95.54%	95.84%	0.9889	27 s
Random Forest	97.63%	97.16%	98.56%	97.86%	0.9977	105 s

XGBoost	98.70%	98.52%	99.13%	98.82%	0.9989	251 s

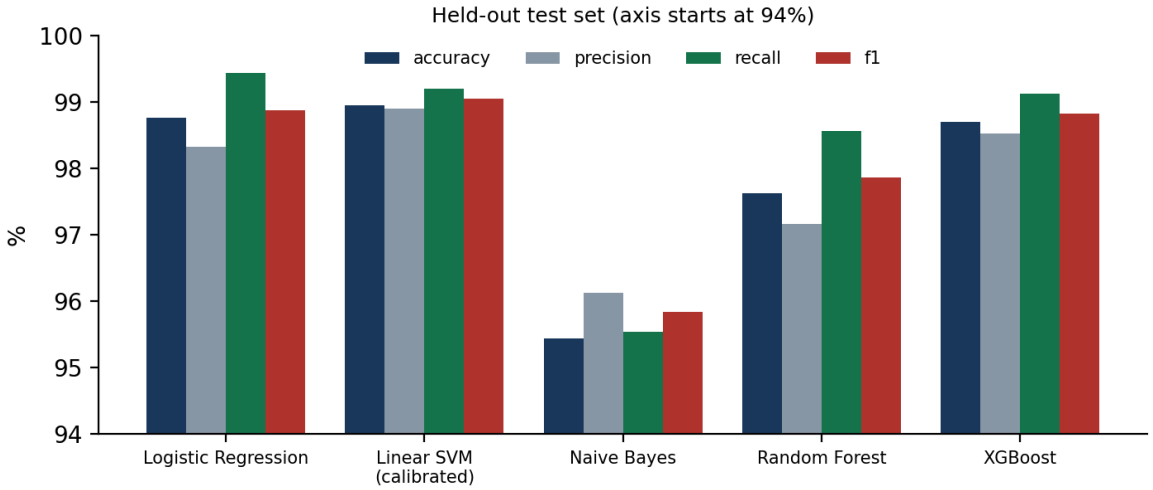


Figure 2. Performance comparison of all five models on the held-out test set inside the platform.

Why Logistic Regression is set as the active model: Technically, the calibrated Linear SVM edged out Logistic Regression by 0.19% on accuracy (98.95% vs. 98.76%), which again is well within single-split noise. However, I kept Logistic Regression as the default active model because it works natively with exact linear SHAP explanations, trains quickly, and outputs naturally calibrated probabilities without needing a wrapper. Naive Bayes (95.44%) and Random Forest (97.63%) trailed behind the linear models and XGBoost.

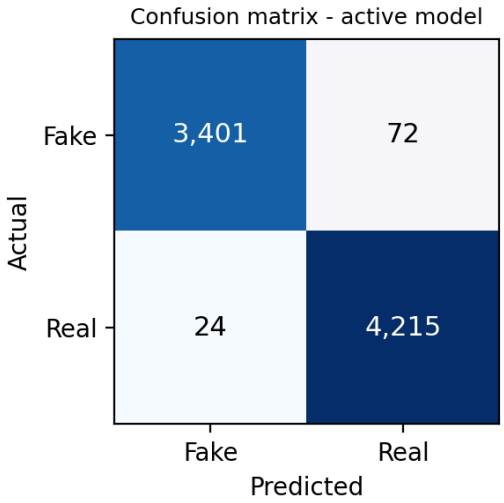


Figure 3. Confusion matrix for the active Logistic Regression model on the 7,712-article test set.

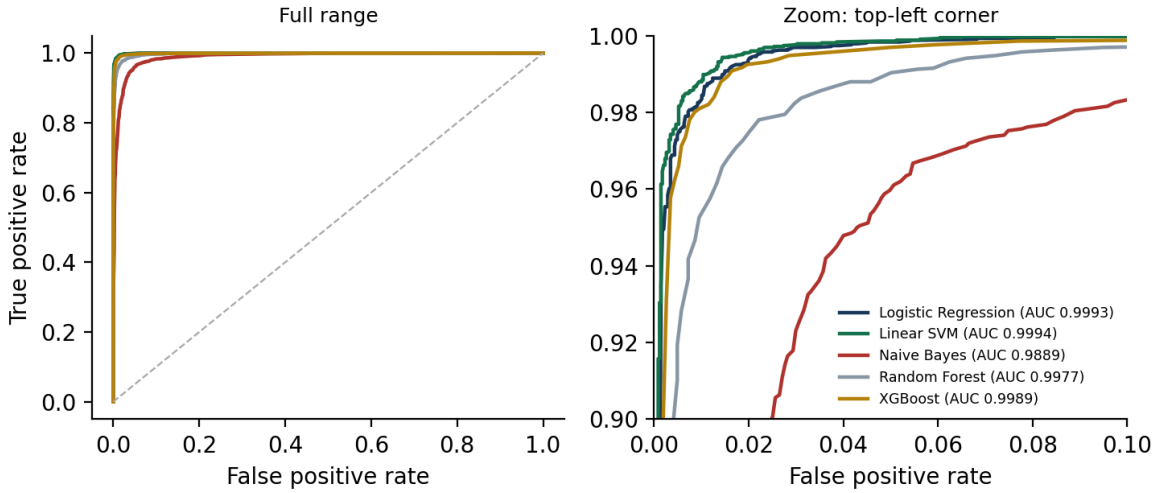


Figure 4. ROC curves on the test set. Logistic Regression, Linear SVM, and XGBoost practically overlap at the top-left, while Random Forest and Naive Bayes sit lower in the zoomed view.

A quick reality check on these numbers: Getting ~98.8% accuracy looks great on paper, but we have to remember that both classes come from just two specific sources. The classifier has gotten really good at telling Reuters journalism apart from partisan blog writing, which is not the same thing as fact-checking a claim. In fact, in our held-out demo set, there is a Fake article that is written in a dry, formal tone, and the model misclassifies it as Real with 98.4% confidence. That kind of overconfident error is the main reason the UI always puts the word-level evidence right next to the score.

6. Explainability

To let users inspect how a prediction was made, the backend supports two explainability methods. The first is LIME, which works with any of the five models: it takes the input article, generates 400 perturbed copies by randomly dropping words, runs them through the classifier, and fits a local ridge regression surrogate to see which words swing the probability the most. While reviewing our early UI screenshots, I noticed LIME was assigning tiny non-zero weights to stop-words and 1-letter fragments that our preprocessor actually deletes before classification. Since those words cannot possibly affect the model, I added a filter to zero out any token discarded by the preprocessor. The second method is SHAP (available when Logistic Regression or XGBoost is active), which computes exact feature attributions against the training-set baseline and maps the stemmed TF-IDF tokens back to the original words in the article.

For the color coding and sign convention: a positive weight means a word pushes the model toward the winning prediction, while a negative weight pushes against it. Inside the article reader view, words that push toward Fake are highlighted in red and words that push toward Real are highlighted in green, with darker shading for stronger weights.

Looking at the explanations tells us a lot about what the model is actually paying attention to. In the civil forfeiture article shown in Figure 5, the word 'said' is the strongest signal pushing toward Real (-0.16 against Fake), because formal news reports constantly attribute quotes using 'said'. Meanwhile, words like 'read' (left over from 'Read more:') and contractions like 'didn' push the score toward Fake. This backs up what we saw in the leakage experiments: the classifier leans heavily on writing style and tone. To help admins keep an eye on this, the admin panel has an Explainability Monitoring tab that tracks the most influential words across all user submissions.

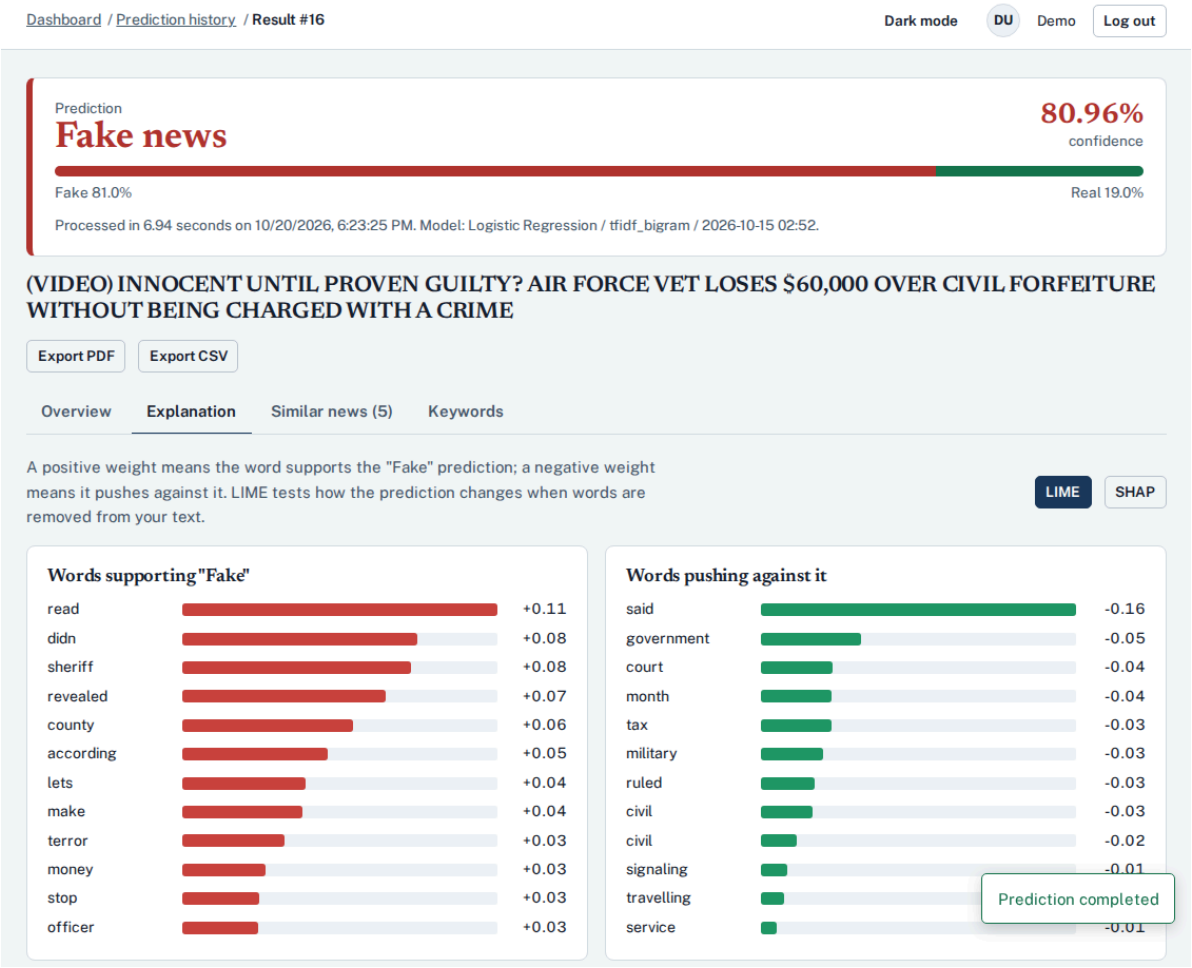


Figure 5. LIME explanation view for a held-out Fake article (predicted Fake at 80.96% confidence), showing the top supporting and opposing words.

7. System design and implementation

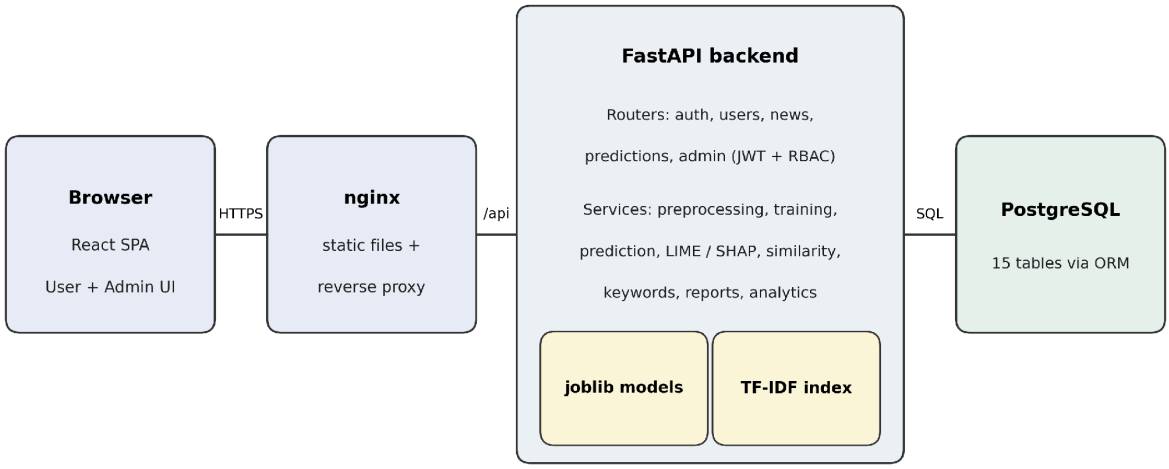


Figure 6. System architecture: React SPA served via Nginx talking to the FastAPI backend and PostgreSQL.

Backend: I built the backend in Python using FastAPI, organized into separate layers for API routers, business-logic services, and SQLAlchemy ORM models. Altogether there are 65 REST endpoints covering everything from auth and user profiles to text analysis, PDF/CSV exports, and admin management (covering all 15 core endpoints required by the project brief). Heavy tasks like importing 40,000 CSV rows or training all five models run in background threads and write their progress percentages to the database so the UI can poll them without timing out.

Database: The app uses PostgreSQL managed through SQLAlchemy. I implemented all ten tables specified in the assignment (Users, NewsArticles, PredictionHistory, Keywords, ExplainabilityResults, SimilarNews, Models, DatasetImports, SystemLogs, and UserSettings) with proper foreign keys and indexes, plus five extra helper tables for training job runs, revoked JWTs, password-reset tokens, global app settings, and the cached keyword dictionary. To guarantee that two models can never be marked active at the same time, I added a partial unique index directly in PostgreSQL rather than relying only on application logic. Duplicate dataset articles are also blocked at the database level using a unique SHA-256 content hash.

Security: User passwords are hashed with bcrypt (using a SHA-256 pre-hash so long passwords do not hit bcrypt's 72-byte limit). Authentication uses signed JWTs, and logging out immediately writes the token ID to a revocation table. Admin routes verify the user's role against the database on every request rather than trusting a stale token claim. I also added login rate-limiting, strict file-upload checks (verifying extensions, file size, and magic bytes for PDF/TXT uploads), environment-based secrets, security headers in Nginx, and a non-root Docker user.

Frontend and Deployment: The frontend is a React single-page application split into a standard user portal and an admin dashboard. It supports light and dark mode, responsive mobile layouts, search and pagination, confirmation modals, and custom 403/404/500 error pages. Everything is wired up with Docker Compose (PostgreSQL, FastAPI, and Nginx), a bootstrap script that loads the CSVs and trains the initial models, and a GitHub Actions CI workflow.

Area	Endpoints (examples)
Authentication	POST /api/auth/register, /login, /logout, /forgot-password, /reset-password; GET /api/auth/me
User	GET/PUT /api/users/profile, PUT /password, POST /profile-image, GET/PUT /settings, GET /dashboard
Analysis	POST /api/news/analyze, /api/news/upload; GET /api/news/similar/{id}
History	GET /api/predictions/history, /{id}, /{id}/export?format=pdf|csv; DELETE; GET /api/explanations/{id}
Administration	users, datasets (preview / upload / progress), news, subjects, models (train / retrain / activate / delete), analytics, explainability, stop-words, settings, logs

8. Testing and defects found

Rather than just clicking around manually, I tested the platform in three stages: (1) an automated pytest suite of 22 API tests covering registration, password hashing, JWT revocation, RBAC permissions, dataset deduplication, model training, single-active-model enforcement, user history isolation, and file uploads—run against both SQLite and PostgreSQL 16; (2) a headless Chromium browser script that logs into both the user and admin interfaces, runs analyses, and captures full-page screenshots; and (3) a visual inspection of every captured screenshot. Running these tests actually caught eight real bugs that I fixed before finalizing the project:

Bug encountered	How I caught it	How I fixed it
PostgreSQL crashed when inserting numpy.float64 values for keyword scores (SQLite had accepted them silently)	Running the API test suite against PostgreSQL 16	Cast all numpy scalars to native Python float() / int() before saving
Querying model status right after a background training job returned a stale status	Automated API test	Added db.refresh() / session reload after the background thread finishes
FastAPI route /users/{id}/{action} accidentally intercepted requests meant for /users/{id}/reset-password	Automated API test	Replaced the wildcard action parameter with explicit sub-routes
Admin analytics page crashed because Chart.js controllers were not registered; the error boundary link also failed outside the router	Headless Chromium browser test	Registered all Chart.js components at startup and used a plain <a> tag on the fallback page
Top navigation bar overflowed horizontally on mobile screens	Browser test at 390 px mobile viewport	Hid the full username/avatar on small screens and truncated long breadcrumbs
LIME assigned small random weights to stop-words that the NLP preprocessor actually strips out	Inspecting the first batch of UI screenshots	Filtered the LIME explanation output against the active preprocessor token list
Train/test split order could vary slightly on PostgreSQL because SELECT without ORDER BY is not deterministic	Code review	Added an explicit ORDER BY id clause before running train_test_split
Live article predictions ignored the optional headline field, even though models were trained on 'title. text' (giving 81.0% instead of 80.65% on our test article)	Comparing UI output against the CLI demo script	Concatenated headline and body identically during inference and added a regression test

What was not tested: While the Dockerfiles and docker-compose.yml are written and the exact same service stack (PostgreSQL 16, FastAPI, and Nginx) was tested end-to-end in a clean Linux environment, I did not build the final

Docker images on this machine. I also did not run load testing for concurrent users or screen-reader accessibility audits.

9. Coverage of the assignment requirements

Requirement	Status	Implementation notes
Preprocessing, tokenisation, stop-words, stemming (6.1)	Done	Includes regex removal of Reuters datelines and publisher boilerplate
Feature extraction justified by experiment (6.2)	Done	Compared BoW, TF-IDF unigrams, and TF-IDF bigrams; Word2Vec / FastText / trigrams not tested
At least three models compared (6.3)	Done	Five models trained and evaluated across accuracy, precision, recall, F1, ROC-AUC, confusion matrix, and ROC curves
LIME / SHAP, positive & negative words, text highlighting (FR-5)	Done	LIME works for all 5 models; exact linear/tree SHAP works for Logistic Regression and XGBoost
Top-5 similar news, keywords, prediction history, PDF/CSV reports (FR-6, 7, 8, 11)	Done	TF-IDF cosine similarity; selectable TF-IDF or RAKE keyword extraction; downloadable PDF and CSV reports
Authentication: register, login, logout, password reset, JWT, RBAC (FR-1)	Done	Password reset links are written to the server log when SMTP is not configured; optional refresh tokens not used
Dataset import with validation, deduplication, progress, versioning (FR-2, 8.3)	Done	Admins can also choose whether to train on all datasets or a single version in Settings
Model management: train, retrain, compare, activate, delete, history (FR-10)	Done	Single active model rule enforced via a PostgreSQL partial unique index
Admin dashboard, analytics, explainability monitoring, system logs (FR-9, 7.2)	Done	Interactive Chart.js visualizations and CSV/PDF summary exports
User dashboard, profile management, preferences, account deletion	Done	Default export format preference works in History; language and email toggles are saved in DB for future use
Light / dark mode, responsive layout, 403 / 404 / 500 error pages	Done	Admins can also set the default theme for new visitors

Docker, Docker Compose, Nginx, CI workflow (9)	Written; stack verified locally	PostgreSQL, FastAPI, and Nginx verified end-to-end outside Docker; CI YAML included; cloud hosting not deployed
TXT and PDF document upload (FR-3)	Done	Extracts text from digital PDFs; scanned image-only PDFs without OCR text are rejected with a clear message

10. Limitations, ethics and future work

- **Publisher style bias —** Because our training data comes from just two contrasting sources, the classifier mostly learns writing style and tone rather than factual accuracy. If you feed it an opinion piece from a mainstream newspaper or a dryly written hoax from another website, accuracy will almost certainly drop. Testing on a multi-source dataset like LIAR or FakeNewsNet is the most obvious next step.
- **Not a fact-checking engine —** The platform has no way to cross-check claims against real-world facts or breaking news events. Satire and op-eds can easily trip it up, which is why the UI includes a clear disclaimer and highlights low-confidence predictions.
- **High-confidence mistakes —** As we saw with our held-out demo set, a linear model can still be 98% confident and completely wrong if a fake article mimics formal news phrasing. Users should always read the highlighted LIME/SHAP words instead of blindly trusting the percentage bar.
- **User privacy —** Articles submitted for analysis are saved to the user's private history so they can review or export them later, but users can delete individual records or wipe their entire account at any time.
- **Where to take this next —** In future iterations, I would like to evaluate cross-domain generalization on external datasets, experiment with fine-tuned transformer embeddings (like DeBERTa or RoBERTa), add sentence-level claim verification, and wire up the multi-language UI strings.

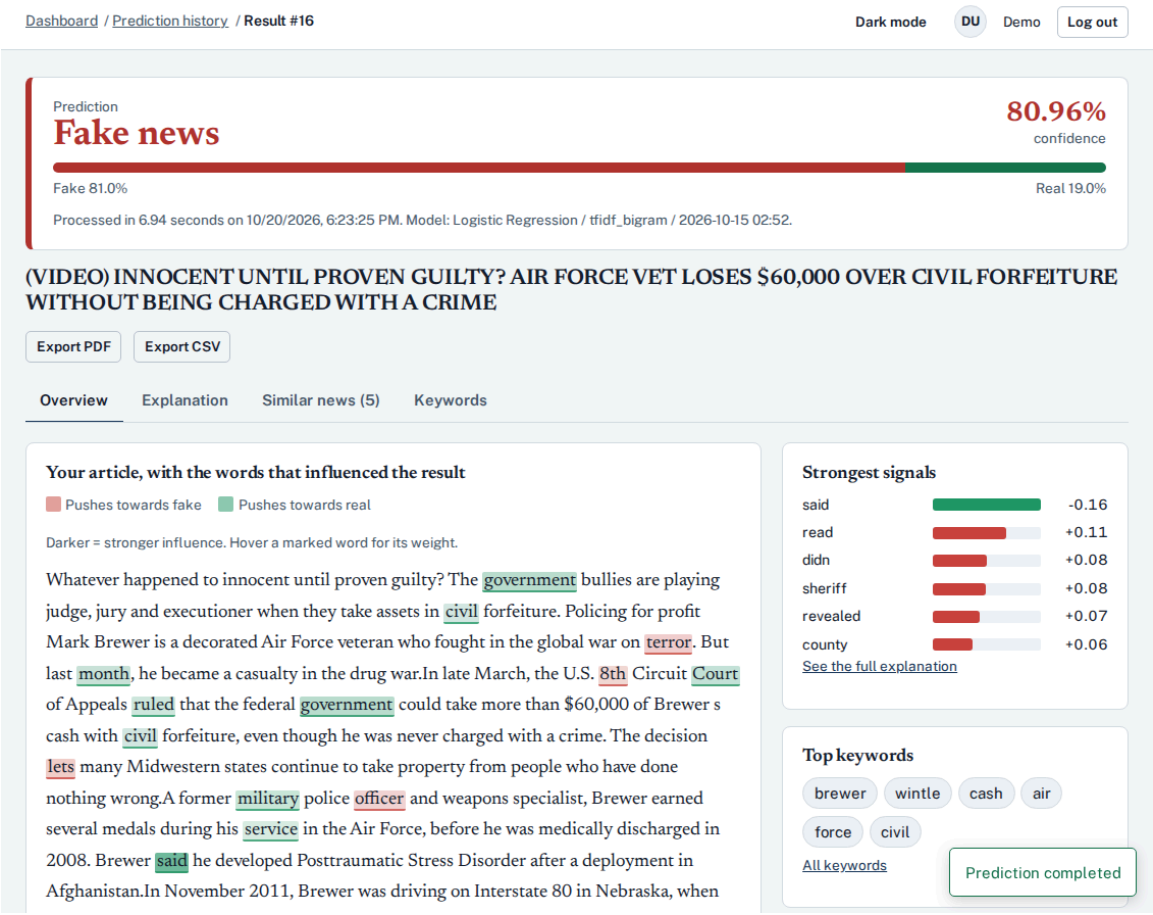
11. Conclusion

Overall, this project puts together a complete, end-to-end fake-news analysis platform: we cleaned a 44,898-row dataset down to 38,559 verified articles, trained and compared five machine learning classifiers reaching up to 98.95% accuracy, built interactive LIME and SHAP word-level explanations, and wrapped everything in a secure React, FastAPI, and PostgreSQL web application. Just as importantly, digging into the data leakage and LIME weights showed us that high accuracy on this benchmark is heavily driven by publisher writing style—a limitation we chose to highlight transparently in both the report and the application's user interface.

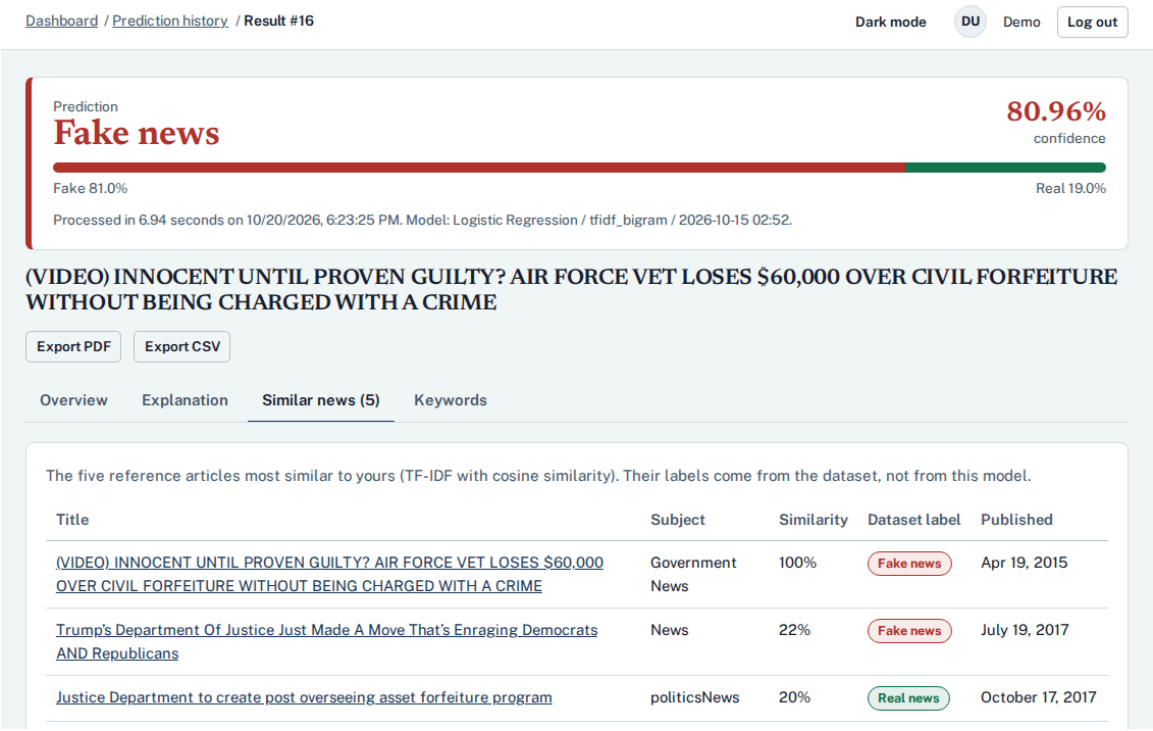
Appendix A. Screens of the application



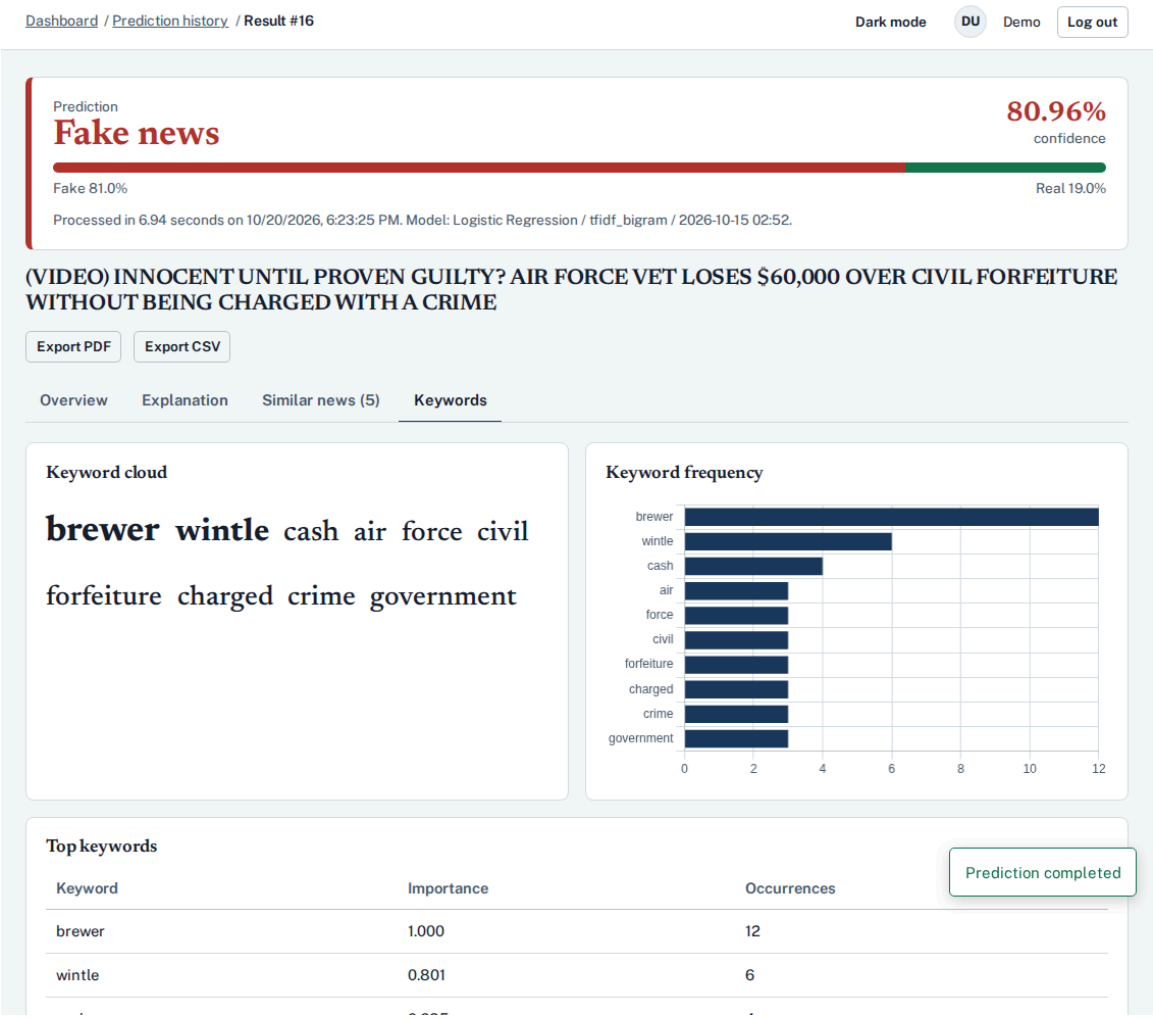
A.1 Analyze page: users can paste article text or upload a TXT/PDF file, with live character and word counters.



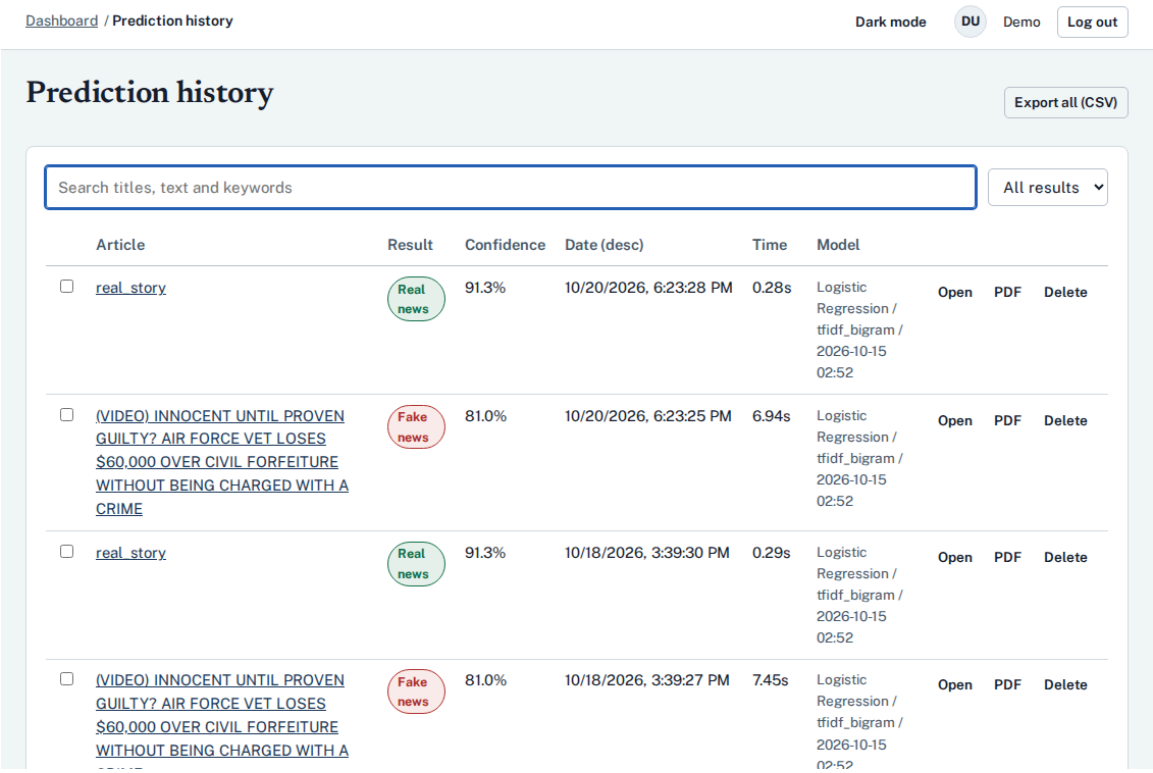
A.2 Result overview: shows the predicted verdict, confidence bar, inline word highlights, top signals, and extracted keywords.



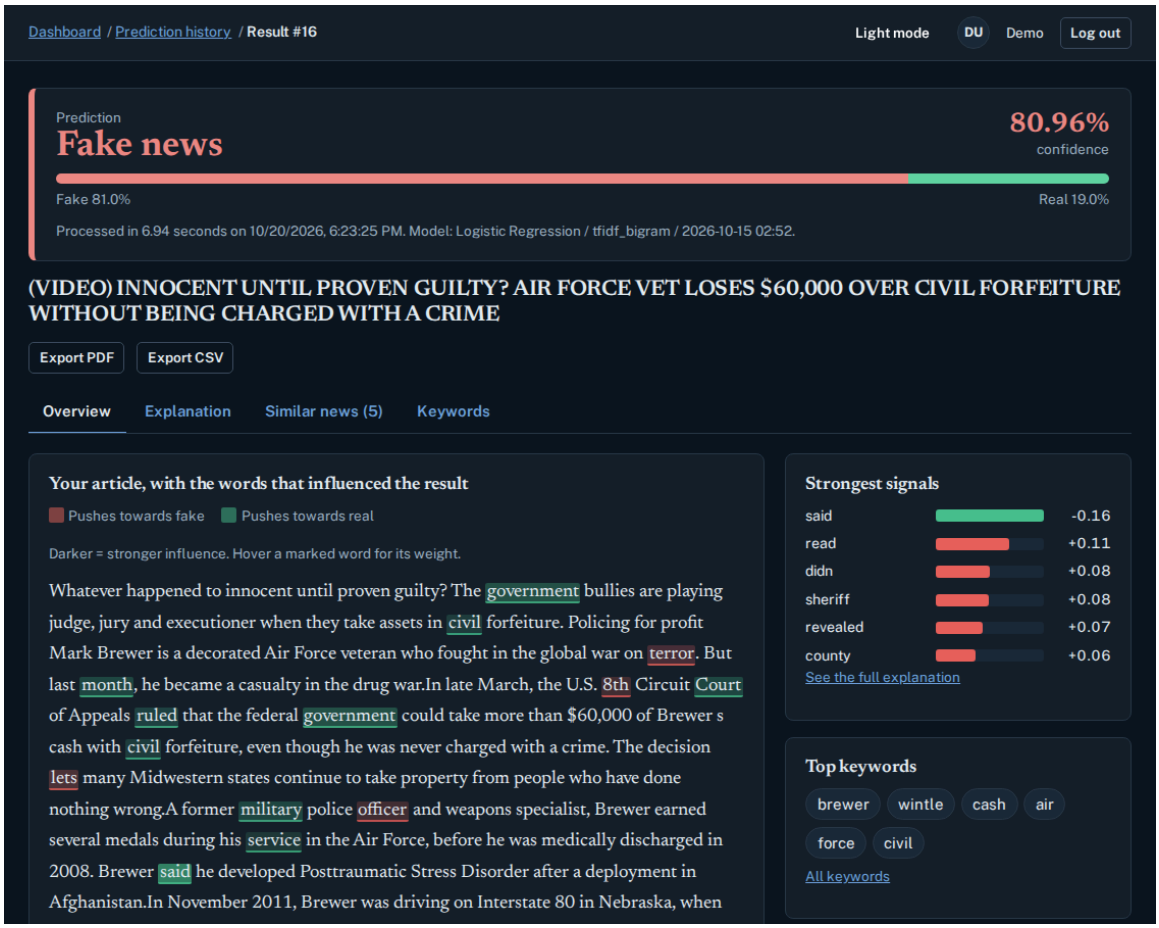
A.3 Similar news tab: lists the five closest articles from the dataset using TF-IDF cosine similarity.



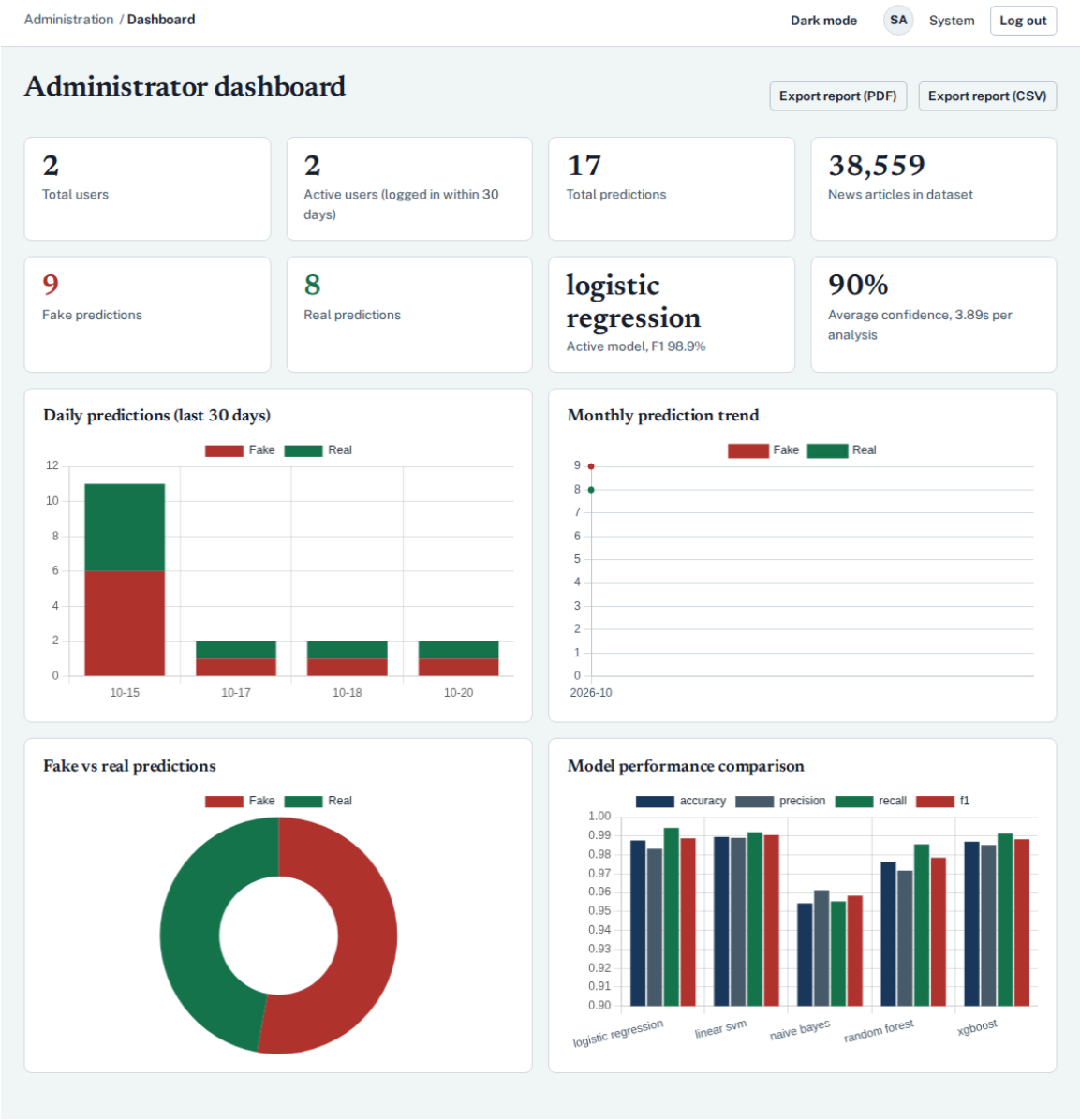
A.4 Keywords tab: visual word cloud, occurrence bar chart, and importance scores.



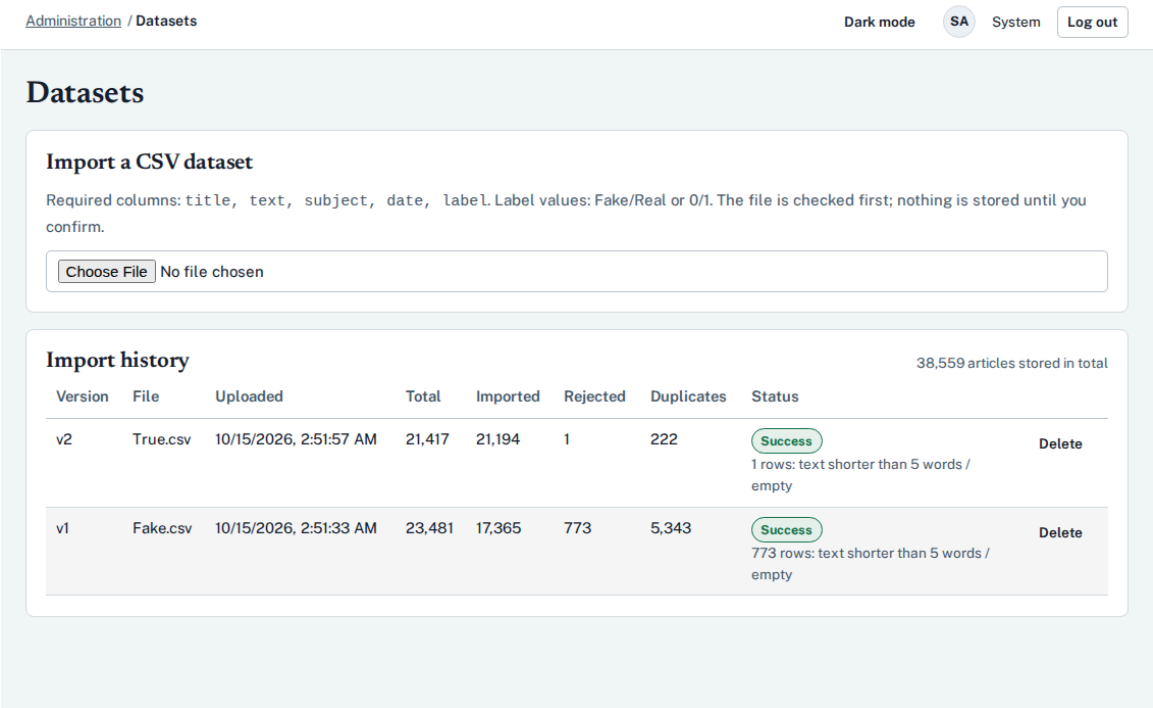
A.5 Prediction history: supports full-text search, filtering, sorting, bulk deletion, and PDF/CSV exports.



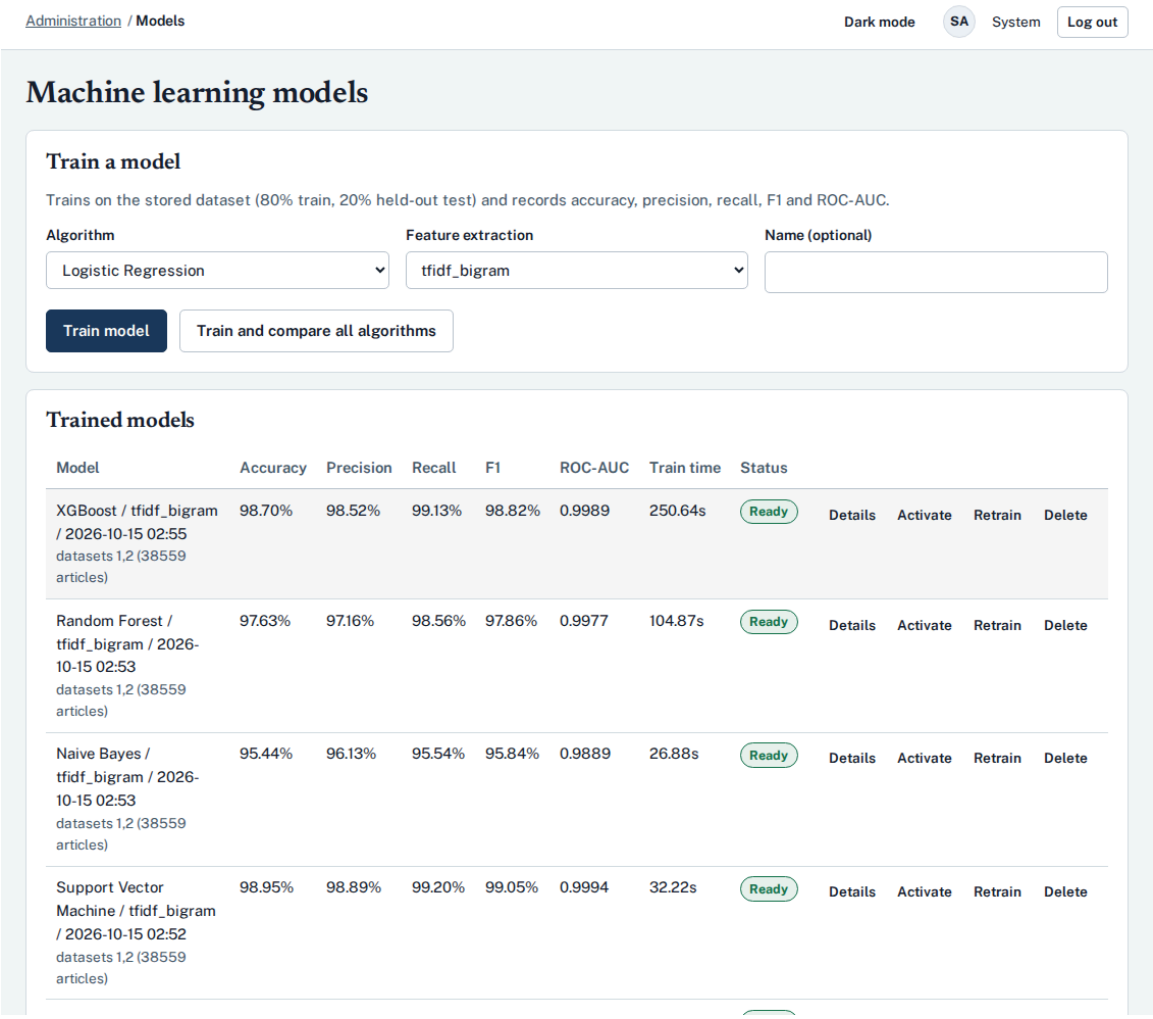
A.6 Dark mode view of the analysis result page.



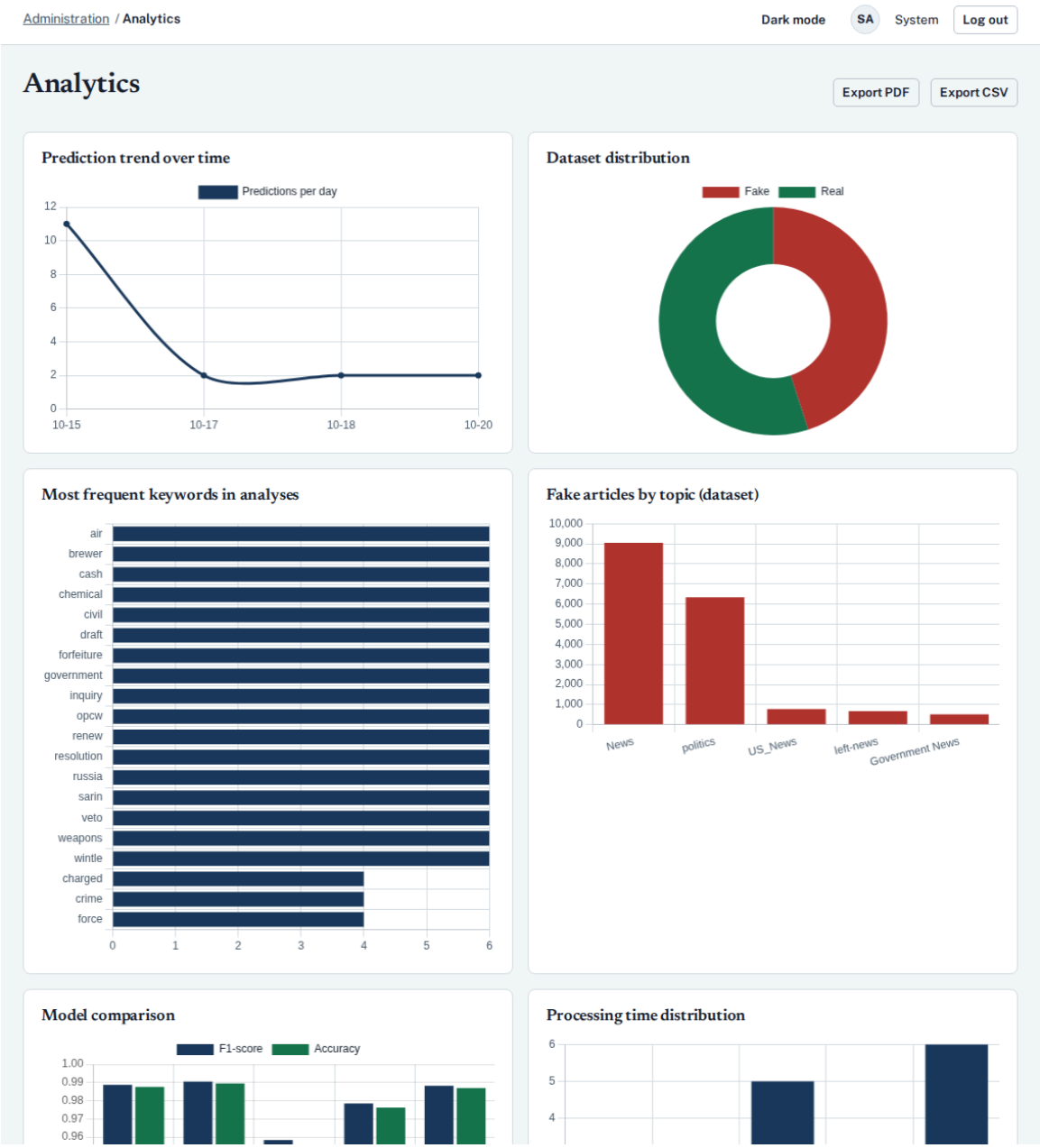
A.7 Administrator dashboard: high-level platform metrics, daily/monthly prediction volume, and model comparison charts.



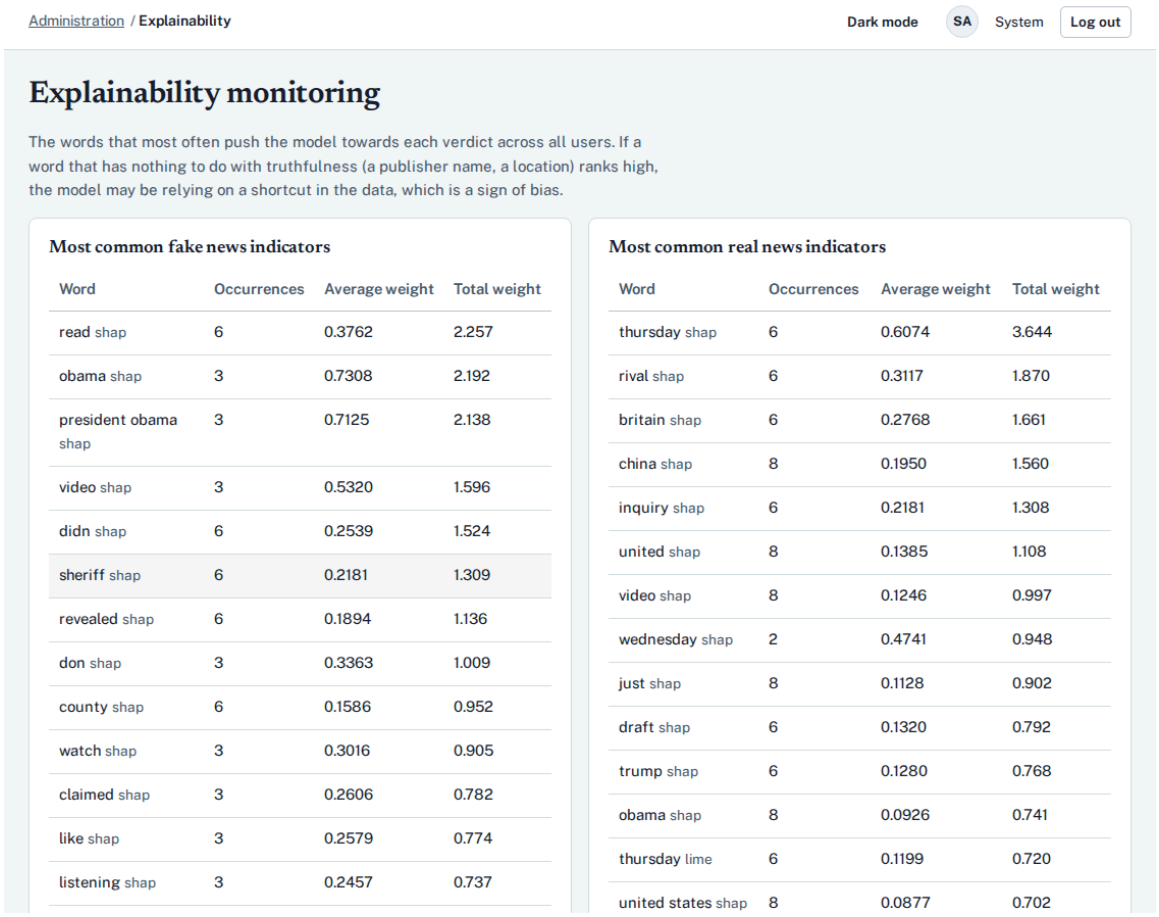
A.8 Dataset management: CSV upload validator and versioned import history showing rejected and duplicate counts.



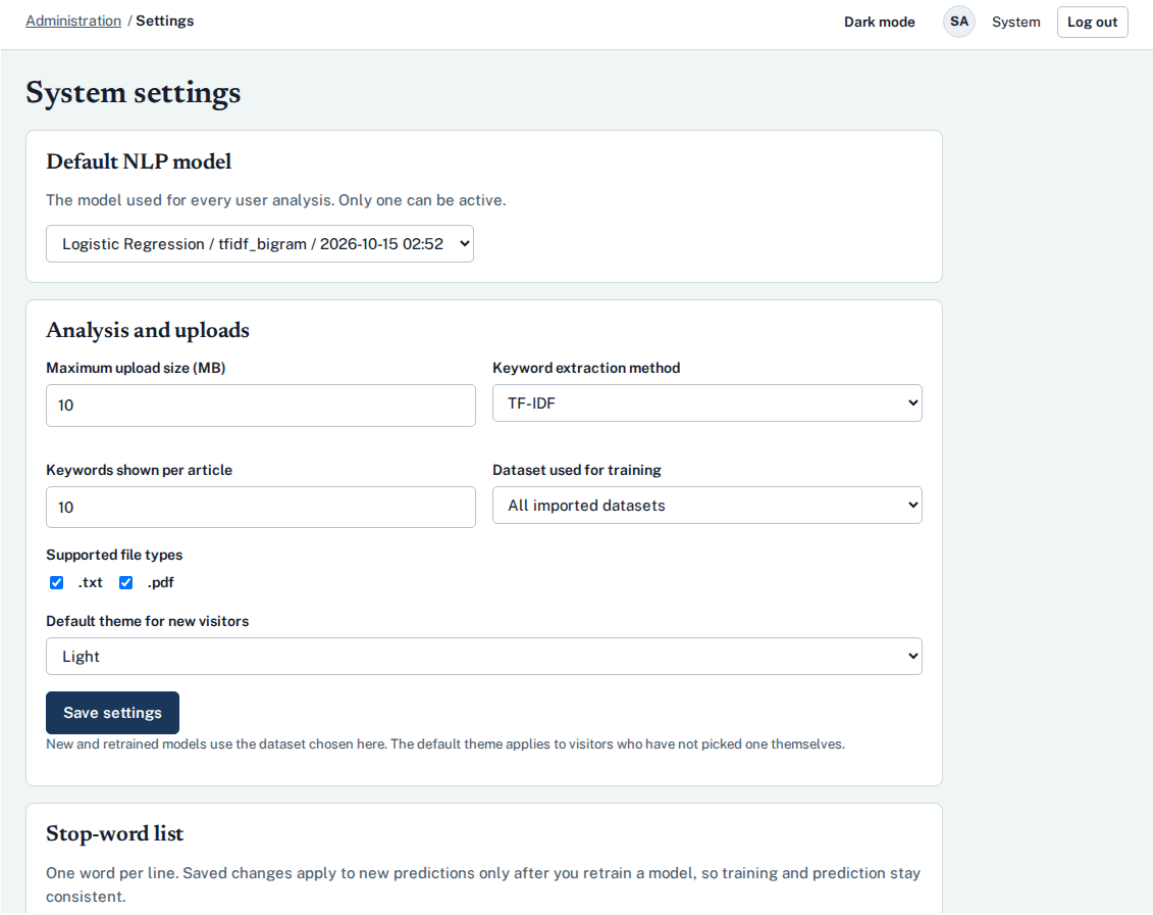
A.9 Model management: train individual algorithms or run a full comparison across all five classifiers.



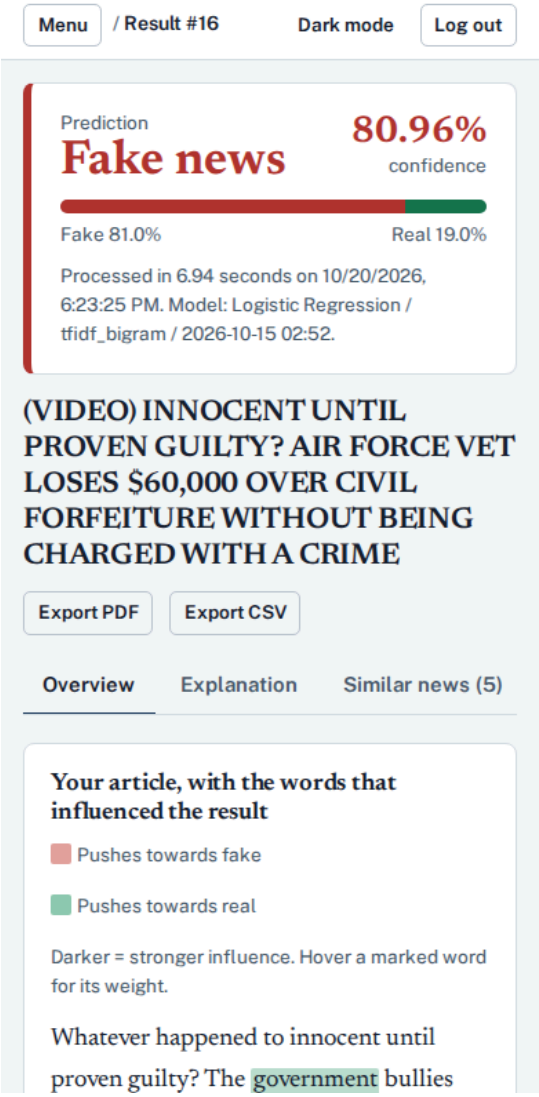
A.10 Admin analytics: prediction trends, class balance, top keywords, fake topics, and processing latency.



A.11 Explainability monitoring: tracks the words that most frequently push predictions toward Fake or Real across all users.



A.12 System settings: configure the active model, upload limits, keyword extractor, default theme, and custom stop-word list.



A.13 Responsive mobile layout tested at a 390 px screen width.

Source code: https://github.com/ABDIRIZAK23HASSAN/fake-news-explainability-platform-220201993

How to run: See README.md for Docker Compose and local development instructions.